Core Parts of the Code

```python
class Tetris:
    # create new instance of Tetris game
    def __init__(self,width,height):
        self.width = width
        self.height = height
        #create an empty grid
        self.grid = [[0 for _ in range(width)] for _ in range(height)]
        self.current_block = self.new_block()
        self.game_over = False
```

The Tetris class is where the game is initiated

1. Starts by creating an empty grid based on the given parameters.
    a. This is the given range where the blocks can be and their boundaries.
2. Creates a new block to start the game.
3. Initiates the game_over variable as False.

```python
#check if the move given is valid
def valid_move(self,block,x,y,rotation):
    #get the block's rotation position
    shape = block.shape[( block.rotation + rotation) % len(block.shape)]

    #for each row in the shape
    for i, row in enumerate(shape):
        #for each cell in the row
        for j, cell in enumerate(row):
            #check all the places a block is present in grid and check if it's not at the edges
            if cell == '0' :
                new_x = block.x + j + x
                new_y = block.y + i + y

                #if block wants to move past the edges or at the bottom move is invalid
                if new_x < 0 or new_x >= self.width or new_y >= self.height:
                    return False
                #if block isn't at the top and slot is filled, move is invalid
                elif new_y > 0 and self.grid[new_y][new_x] != 0:
                    return False
                #if block is at the top and slot is filled then the game is over.
                elif new_y == 0 and self.grid[new_y][new_x] != 0:
                    self.game_over = True
                    return False

    #block is in bounds and not touching another block, move is valid.
    return True
```

This is where the block's movements are checked. It checks whether the given move is valid.

Give the function an X and Y to where the block wants to move, add it to the original position of each cell in the block. Check if that position is in bounds, at another block, or at the top of the boundary. If block is in bounds and at the top of the game but is at another block, the game ends.

Shape get's which shape the block is, and which position it is in. Each block holds a list of it's possible positions.

```python
[ #LONG BAR
    #pos 1
    ['.....',
    '.....',
    '.....',
    '0000.',
    '.....'
    ],

    #pos 2
    [
     '.....',
     '..0..',
     '..0..',
     '..0..',
     '..0..'
     ]

    ], # end Long Bar
```

```
#CONTINUOUS PRESS ACTIONS

keys = pygame.key.get_pressed()

if not game.game_over:
    move_time += dtime
    if move_time > move_delay:

        #if user holds down left, keep moving block left.
        if keys[pygame.K_LEFT]:
            if game.valid_move(game.current_block, -1, 0, 0):
                game.current_block.x -= 1

        #if user holds down right, keep moving block right.
        if keys[pygame.K_RIGHT]:
            if game.valid_move(game.current_block, 1, 0, 0):
                game.current_block.x += 1

        #if user holds down right, move block down faster.
        if keys[pygame.K_DOWN]:
            if game.valid_move(game.current_block, 0, 1, 0):
                game.current_block.y += 1
        move_time = 0
```

This is where the block is controlled.

If the game is not over then read inputs and check if they are valid moves. If they are valid then move the block in that direction. Move_time and move_delay are to prevent the blocks from moving too fast and intermittently move the blocks. Dtime is the change in time since the last frame.